

```
chmod +x filename.sh
./filename.sh
```

```
gcc program.c -o output
./output
```

1. Write a bash script to print all even numbers from 1 to 50.

```
#!/bin/bash
echo "Even numbers from 1 to 50"
for i in {1..50}
do
    if ((i%2==0))
    then
        echo "$i"
    fi
done
```

Run :

```
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ chmod even.odd.sh
```

```
piyush@ChirlePiyush:~$ bash even.odd.sh
```

output:

Even numbers from 1 to 50

```
2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
```

40
42
44
46
48
50

2. Write a program where the main function accepts integers to be sorted, uses the FORK system call to create a child process, with the parent process sorting the integers using sorting algorithm and waiting for the child process using the WAIT system call, while the child process sorts the integers using any sorting algorithm, and demonstrate the zombie and orphan process states during execution.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```
void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
void selectionSort(int arr[], int n) {
    int i, j, min_index, temp;
    for(i = 0; i < n - 1; i++) {
        min_index = i;
        for(j = i + 1; j < n; j++) {
            if(arr[j] < arr[min_index])
                min_index = j;
        }
        temp = arr[i];
        arr[i] = arr[min_index];
```

```

    arr[min_index] = temp;
}
}

void printArray(int arr[], int n) {
    for(int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main(int argc, char *argv[]) {
    if(argc < 2) {
        printf("Usage: %s <numbers>\n", argv[0]);
        return 1;
    }

    int n = argc - 1;
    int arr[n];

    for(int i = 0; i < n; i++)
        arr[i] = atoi(argv[i + 1]);

    pid_t pid = fork();

    if(pid == 0) {
        printf("\nChild Process\n");
        selectionSort(arr, n);
        printArray(arr, n);
        exit(0);
    } else {
        bubbleSort(arr, n);
        printArray(arr, n);
        wait(NULL);
    }

    return 0;
}

```

Output:

```

Usage: ./output <numbers>
piyush@ChirlePiyush:~$ ./output 1 2 3
1 2 3

```

```

Child Process
1 2 3

```

```
piyush@ChirlePiyush:~$ gcc fork.sorting.c -o output
piyush@ChirlePiyush:~$ ./ output 1 2 4 7 8 4
-bash: ./: Is a directory
piyush@ChirlePiyush:~$ ./output 1 2 4 7 8 4
1 2 4 4 7 8
```

Child Process
1 2 4 4 7 8

3. Write a script to take user input using read and display it using echo. & Write a script to find the largest of two numbers entered by the user.

```
#!/bin/bash
echo "Enter your name:"
read name
echo "Hello, $name!"
```

Run:
chmod +x *
Bash user_input.sh

Output:
piyush@ChirlePiyush:~\$ bash user_input.sh
Enter your name:
piyush
Hello, piyush!

```
#!/bin/bash
echo "Enter first number:"
read num1
echo "Enter second number:"
read num2
```

```
if [ $num1 -gt $num2 ]
then
echo "$num1 is greater"
elif [ $num1 -lt $num2 ]
then
echo "$num2 is greater"
else
```

```
echo "Equal"
fi
```

Run 👍

```
piyush@ChirlePiyush:~$ nano large.no.sh
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ bash large.no.sh
```

output:

Enter first number:

2

Enter second number:

7

7 is greater

4. Write a program to simulate any one CPU scheduling algorithm (FCFS, SJF)

```
#include <stdio.h>
```

```
void fcfs(int bt[], int n) {
    int wt[10], tat[10];
    wt[0] = 0;
```

```
    for(int i=1; i<n; i++)
        wt[i] = wt[i-1] + bt[i-1];
```

```
    for(int i=0; i<n; i++) {
        tat[i] = wt[i] + bt[i];
        printf("P%d %d %d %d\n", i+1, bt[i], wt[i], tat[i]);
    }
}
```

```
int main() {
    int n, bt[10];
    scanf("%d", &n);
```

```
    for(int i=0; i<n; i++)
        scanf("%d", &bt[i]);
```

```
    fcfs(bt, n);
    return 0;
```

```
}
```

```
piyush@ChirlePiyush:~$ gcc fcfs.c -o output
```

```
piyush@ChirlePiyush:~$ ./output
```

```
4
```

```
3
```

```
3
```

```
2
```

```
4
```

```
P1 3 0 3
```

```
P2 3 3 6
```

```
P3 2 6 8
```

```
P4 4 8 12
```

```
*****
```

5. Write a script to check whether a file exists using test and display its content using cat.

```
#!/bin/bash
```

```
echo "Enter filename:"
```

```
read file
```

```
if [ -f "$file" ]
```

```
then
```

```
    echo "File exists"
```

```
    cat "$file"
```

```
else
```

```
    echo "File not found"
```

```
fi
```

```
piyush@ChirlePiyush:~$ nano file.exists.sh
```

```
piyush@ChirlePiyush:~$ chmod +x *
```

```
piyush@ChirlePiyush:~$ bash file.exists.sh
```

```
Enter filename:
```

```
fcfs.c
```

```
File exists
```

```
#include <stdio.h>
```

```
void fcfs(int bt[], int n) {
```

```
    int wt[10], tat[10];
```

```
    wt[0] = 0;
```

```
    for(int i=1; i<n; i++)
```

```
        wt[i] = wt[i-1] + bt[i-1];
```

```

for(int i=0;i<n;i++) {
    tat[i]=wt[i]+bt[i];
    printf("P%d %d %d %d\n",i+1,bt[i],wt[i],tat[i]);
}
}

```

```

int main() {
    int n, bt[10];
    scanf("%d",&n);

    for(int i=0;i<n;i++)
        scanf("%d",&bt[i]);

```

```

    fcfs(bt,n);
    return 0;
}

```

```

piyush@ChirlePiyush:~$ bash file.exists.sh
Enter filename:
piyush.sh
File exists
#!/bin/bash

```

```

echo "piyush"
piyush@ChirlePiyush:~$ bash file.exists.sh
Enter filename:
om.sh
File not found

```

6. Write a program to solve the Producer-Consumer problem utilizing counting semaphores along with mutex or binary semaphores for synchronization

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

```

```

#define SIZE 5
int buffer[SIZE], in=0, out=0;

```

```

sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg){
    for(int i=0;i<5;i++){
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in]=i;
        printf("Produced %d\n",i);
        in=(in+1)%SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
}

void* consumer(void* arg){
    for(int i=0;i<5;i++){
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        printf("Consumed %d\n",buffer[out]);
        out=(out+1)%SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
}

int main(){
    pthread_t p,c;
    sem_init(&empty,0,SIZE);
    sem_init(&full,0,0);
    pthread_mutex_init(&mutex,NULL);

    pthread_create(&p,NULL,producer,NULL);
    pthread_create(&c,NULL,consumer,NULL);

    pthread_join(p,NULL);
    pthread_join(c,NULL);
}

```

File not found


```
piyush@ChirlePiyush:~$ nano producer.consumer.c
piyush@ChirlePiyush:~$ gcc producer.consumer.c -o output
piyush@ChirlePiyush:~$ ./output
Produced 0
Produced 1
Produced 2
Consumed 0
Consumed 1
Produced 3
Produced 4
Consumed 2
Consumed 3
Consumed 4
```

7. Write a script to list files using ls and create a new file using touch. & Write a script to find the factorial of a number.

```
#!/bin/bash
echo "Enter number:"
read n

fact=1

for ((i=1;i<=n;i++))
do
    fact=$((fact*i))
done

echo "Factorial of $n is $fact"
```

output:

```
piyush@ChirlePiyush:~$ nano factorial.sh
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ bash factorial.sh
Enter number:
5
Factorial of 5 is 120
```

Q8.) Write a script to check whether a file exists using test and display its content using cat.

```
#!/bin/bash
echo "Enter filename:"
read filename

echo "Enter word to search:"
read word

if [ -f "$filename" ]
then
    echo "File exists."
    echo "Searching for '$word'..."
    grep "$word" "$filename"
    echo "Full content:"
    cat "$filename"
else
    echo "File does not exist."
fi
```

Run and output

```
piyush@ChirlePiyush:~$ nano grep.cat.sh
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ ./grep.cat.sh
Enter filename:
factorial.sh
Enter word to search:
for
File exists.
Searching for 'for'...
for ((i=1;i<=n;i++))
Full content:
#!/bin/bash
echo "Enter number:"
read n

fact=1

for ((i=1;i<=n;i++))
do
    fact=$((fact*i))
```

```
done
```

```
echo "Factorial of $n is $fact"
```

```
*****
```

Q9) Write a script to compare two files using diff and display a message using echo based on result.

```
#!/bin/bash
echo "Enter first file:"
read f1

echo "Enter second file:"
read f2

if [ -f "$f1" ] && [ -f "$f2" ]
then
    diff "$f1" "$f2" > /dev/null

    if [ $? -eq 0 ]
    then
        echo "Files are identical"
    else
        echo "Files are different"
        diff "$f1" "$f2"
    fi
else
    echo "File not found"
fi
```

Run and output 👍

```
piyush@ChirlePiyush:~$ nano diffrent.same.sh
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ bash diffrent.same.sh
Enter first file:
factorial.sh
Enter second file:
cat.sh
File not found
piyush@ChirlePiyush:~$ bash diffrent.same.sh
Enter first file:
factorial.sh
Enter second file:
```

```

file.exists.sh
Files are different
2,3c2,3
< echo "Enter number:"
< read n
---
> echo "Enter filename:"
> read file
5,12c5,11
< fact=1
<
< for ((i=1;i<=n;i++))
< do
< fact=$((fact*i))
< done
<
< echo "Factorial of $n is $fact"
---
> if [ -f "$file" ]
> then
> echo "File exists"
> cat "$file"
> else
> echo "File not found"
> fi

```

10) Write a script to display file content using cat and reverse content using tac.

```

#!/bin/bash
echo "Enter filename:"
read file

if [ -f "$file" ]
then
echo "Content:"
cat "$file"

echo "Reverse:"
tac "$file"
else
echo "File not found"
fi

```

```
piyush@ChirlePiyush:~$ nano cat.tac.sh
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ bash cat.tac.sh
Enter filename:
```

```
factorial.sh
```

```
Content:
```

```
#!/bin/bash
```

```
echo "Enter number:"
```

```
read n
```

```
fact=1
```

```
for ((i=1;i<=n;i++))
```

```
do
```

```
fact=$((fact*i))
```

```
done
```

```
echo "Factorial of $n is $fact"
```

```
Reverse:
```

```
echo "Factorial of $n is $fact"
```

```
done
```

```
fact=$((fact*i))
```

```
do
```

```
for ((i=1;i<=n;i++))
```

```
fact=1
```

```
read n
```

```
echo "Enter number:"
```

```
#!/bin/bash
```

```
*****
```

11) Write a script to check eligibility of person for voting.

```
#!/bin/bash
echo "Enter age:"
read age

if [ "$age" -ge 18 ]
then
    echo "Eligible to vote"
else
    echo "Not eligible"
fi
```

Run and output:

```
piyush@ChirlePiyush:~$ nano voting.sh
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ bash voting.sh
Enter age:
20
Eligible to vote
```

12) Write a script to check whether a number is positive, negative, or zero.

```
#!/bin/bash
echo "Enter number:"
read num

if [ "$num" -gt 0 ]
then
    echo "Positive"
elif [ "$num" -lt 0 ]
then
    echo "Negative"
else
    echo "Zero"
fi
```

```
Eligible to vote
piyush@ChirlePiyush:~$ nano positive.negative.zero.sh
piyush@ChirlePiyush:~$ bash positive.negative.zero.sh
Enter number:
4
Positive
```

13) Write a script to replace a word in a file using sed and display updated content using cat.

```
#!/bin/bash
echo "Enter filename:"
read filename
echo "Enter word to replace:"
read oldword
echo "Enter new word:"
read newword

if [ -f "$filename" ]
then
    echo "File exists."
    sed -i "s/$oldword/$newword/g" "$filename"
    echo "Word replaced successfully."
    echo "Updated file content:"
    cat "$filename"
else
    echo "File does not exist."
fi
```

Run and output:

```
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ bash replace.sed.sh
Enter filename:
    positive.negative.zero.sh
Enter word to replace:
if
Enter new word:
or
File exists.
Word replaced successfully.
Updated file content:
#!/bin/bash
echo "Enter number:"
read num

or [ "$num" -gt 0 ]
```

```
then
  echo "Positive"
elif [ "$num" -lt 0 ]
then
  echo "Negative"
else
  echo "Zero"
fi
piyush@ChirlePiyush:~$
```

14) Addition of Two Numbers

```
#!/bin/bash
echo "Enter first number:"
read num1
echo "Enter second number:"
read num2

sum=$((num1 + num2))

echo "Sum of $num1 and $num2 is $sum"
```

Run nd output:

```
piyush@ChirlePiyush:~$ ./add.two.num.sh
Enter first number:
3
Enter second number:
9
Sum of 3 and 9 is 12
piyush@ChirlePiyush:~$
```

15) Check Number > 10

```
#!/bin/bash
echo "Enter a number:"
read num
```



```
if [ "$num" -gt 10 ]
then
  echo "The number is greater than 10."
else
  echo "The number is less than or equal to 10."
Fi
```

Run and output:

```
iyush@ChirlePiyush:~$ nano gt.than.ten.sh
piyush@ChirlePiyush:~$ ./gt.than.ten.sh
Enter a number:
55
The number is greater than 10.
```

16) Print Numbers using FOR Loop

```
#!/bin/bash
for i in {1..10}
do
  echo "Number: $i"
Done
```

```
piyush@ChirlePiyush:~$ nano for.loop.print.sh
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ bash for.loop.print.sh
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
Number: 6
Number: 7
Number: 8
Number: 9
Number: 10
piyush@ChirlePiyush:~$
```


17) Write a script to print numbers using a while loop and increment using arithmetic operations.

```
#!/bin/bash
i=1
while [ $i -le 10 ]
do
    echo "Number: $i"
    i=$((i + 1))
done
```

Run output:

```
piyush@ChirlePiyush:~$ nano while.loop.sh
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ nano while.loop.sh
piyush@ChirlePiyush:~$ ./while.loop.sh
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
Number: 6
Number: 7
Number: 8
Number: 9
Number: 10
piyush@ChirlePiyush:~$
```

18) Write a script to check even or odd using conditionals and display result using echo.

```
#!/bin/bash
echo "Enter a number:"
read num

if [ $((num % 2)) -eq 0 ]
```

```

then
echo "The number $num is Even."
else
echo "The number $num is Odd."
fi

```

Run and output:

```

piyush@ChirlePiyush:~$ nano even.odd.conditional.sh
piyush@ChirlePiyush:~$ chmod +x *
piyush@ChirlePiyush:~$ ./even.odd.conditional.sh
Enter a number:
5
The number 5 is Odd.

```

19) Write a bash script to calculate area of circle.

```

#!/bin/bash
echo "Enter radius:"
read r
area=$((22/7 * r * r))
echo "Area of circle is: $area"

```

Run and output:

```

piyush@ChirlePiyush:~$ nano area.circle.sh
piyush@ChirlePiyush:~$ bash area.circle.sh
Enter radius:
6
Area of circle is: 108
piyush@ChirlePiyush:

```


20) Write a program to solve the Reader-Writer problem with a focus on giving priority to readers for synchronization.

```

#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t wrt;
pthread_mutex_t mutex;
int readcount = 0;

void *reader(void *arg)
{
    int id = *(int *)arg;

    pthread_mutex_lock(&mutex);
    readcount++;
    if(readcount == 1)
        sem_wait(&wrt);
    pthread_mutex_unlock(&mutex);

    printf("Reader %d is reading\n", id);
    sleep(1);

    pthread_mutex_lock(&mutex);
    readcount--;
    if(readcount == 0)
        sem_post(&wrt);
    pthread_mutex_unlock(&mutex);

    return NULL;
}

void *writer(void *arg)
{
    int id = *(int *)arg;

    sem_wait(&wrt);
    printf("Writer %d is writing\n", id);
    sleep(2);
    sem_post(&wrt);

    return NULL;
}

int main()
{
    pthread_t r[3], w[2];
    int id[5];

```

```

sem_init(&wrt, 0, 1);
pthread_mutex_init(&mutex, NULL);

for(int i = 0; i < 3; i++)
{
    id[i] = i + 1;
    pthread_create(&r[i], NULL, reader, &id[i]);
}

for(int i = 0; i < 2; i++)
{
    id[i] = i + 1;
    pthread_create(&w[i], NULL, writer, &id[i]);
}

for(int i = 0; i < 3; i++)
    pthread_join(r[i], NULL);

for(int i = 0; i < 2; i++)
    pthread_join(w[i], NULL);

sem_destroy(&wrt);
pthread_mutex_destroy(&mutex);

return 0;
}

```

Run output:-

```

piyush@ChirlePiyush:~$ nano reader.writer.problem.sync.sh
piyush@ChirlePiyush:~$ mv reader.writer.problem.sync.sh reader.writer.problem.sync.c
piyush@ChirlePiyush:~$ gcc reader.writer.problem.sync.c -o output
piyush@ChirlePiyush:~$ ./output
Reader 1 is reading
Reader 3 is reading
Reader 2 is reading
Writer 1 is writing
Writer 2 is writing
piyush@ChirlePiyush:~$

```

21) Write a program for Deadlock Avoidance Algorithm..

```

#include <stdio.h>

int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m], finish[n], safeSeq[n];

    printf("\nEnter Allocation Matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("\nEnter Max Matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("\nEnter Available Resources:\n");
    for(int i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    for(int i = 0; i < n; i++)
        finish[i] = 0;

    int count = 0;

    while(count < n) {
        int found = 0;
        for(int i = 0; i < n; i++) {
            if(finish[i] == 0) {
                int j;
                for(j = 0; j < m; j++) {
                    if(need[i][j] > avail[j])
                        break;
                }
            }

            if(j == m) {
                for(int k = 0; k < m; k++)

```

```
avail[k] += alloc[i][k];
```

```
safeSeq[count++] = i;
```

```
finish[i] = 1;
```

```
found = 1;
```

```
}
```

```
}
```

```
}
```

```
if(found == 0) {
```

```
printf("\nSystem is NOT in safe state!\n");
```

```
return 0;
```

```
}
```

```
}
```

```
printf("\nSystem is in SAFE state.\nSafe sequence: ");
```

```
for(int i = 0; i < n; i++)
```

```
printf("P%d ", safeSeq[i]);
```

```
printf("\n");
```

```
return 0;
```

```
}
```